

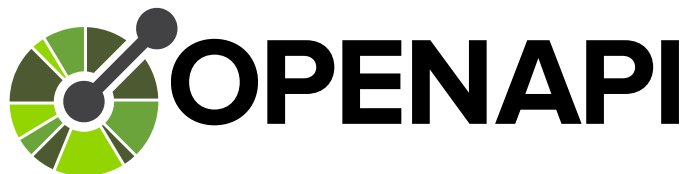
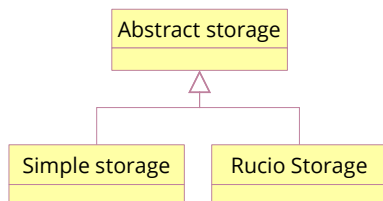


SKAO Regional Centre United Kingdom

IVOA Execution Broker

Progress report

Dave Morris
Manchester
University



June 2026

Dave Morris
dave.morris@manchester.ac.uk



TAP is a set of endpoints that perform operations on tables.

- /sync
- /async
- /tables

The simplest of which is /sync

POST an ADQL query to /sync and the service will execute it



ExecutionBroker is a set of endpoints that manage execution sessions.

- `/direct` Direct execution (POST, redirect)
- `/requests` Execution request (POST, redirect)
- `/offers/{uuid}` Offers response (GET)
- `/sessions/{uuid}` Session status (GET)

The simplest of which is `/direct`

POST a task description to `/direct` and the service will execute it



The Execution Broker data model describes a re-usable set of components.

Executables:

- Docker container
- Jupyter notebook
- etc

Data resources:

- Simple HTTP resource
- Amazon S3 object
- Rucio object
- etc

Storage resources:

- Filesystem resource
- etc

Compute resources:

- Simple compute resource
- etc



A simple request to run a Docker container:

```
executable:
  kind: "http://purl/docker-container"
  image:
    locations:
      - "ghcr.io/zarquan/heliophorus-androcles"
    digest: "sha256:0dfe....4df3"
```

POST this to the /direct endpoint and the platform will run the container.

A simple request to run a Jupyter notebook:

```
executable:  
  kind: "http://purl/jupyter-notebook"  
  notebook:  
    location: "http://....example.jpnb"
```

POST this to the /direct endpoint and the platform will run the notebook.



Posting a task with just the executable assumes that the data you want to work on is already available on the platform.

If not, then we can add a description of the data resource we want to use.

executable:

```
kind: "http://purl/docker-container"
```

```
image:
```

```
locations:
```

```
- "ghcr.io/zarquan/heliophorus-androcles"
```

```
digest: "sha256:0dfe....4df3"
```

data:

```
- kind: http://purl/simple-data-resource
```

```
location: "http://....example.dat"
```



To complete the task description, we need to add a volume mount to make the data available in the right place.

```
executable:
  kind: "http://purl/docker-container"
  image:
    locations:
      - "ghcr.io/zarquan/heliophorus-androcles"
    digest: "sha256:0dfe....4df3"
compute:
  kind: : http://purl/simple-compute-resource
  volumes:
    - kind: http://purl/simple-volume-mount
      path: /input
      mode: READONLY
      resource: example-data
data:
  - kind: http://purl/simple-data-resource
    meta:
      name: example-data
      location: "http://....example.dat"
```

POST this to the /direct endpoint and the platform will download the data to the platform, mount it in the right location, and then run the container.



The underlying pattern is based on Openstack, Docker, and Kubernetes

- The kind identifier for types comes from Kubernetes.
- The volume mounts come from Openstack and Docker.
- The container image details come from Docker.

But the model is abstracted away from the specific platforms to be able to manage tasks on a wide range of different platforms.

- Openstack
- Docker
- Kubernetes
- Slurm
- Dirac
- Amazon AWS
- Digital Ocean
- etc ..

We have a Python module that helps to create and send requests.

```

return ExecutionRequest(
    executable=DockerContainer(
        image=DockerImageSpec(
            locations=[ANDROCLES_IMAGE],
            digest=ANDROCLES_DIGEST,
        ),
        command=["md5sum", "json"],
    ),
    data=[
        SimpleDataResource(
            meta=ComponentMetadata(name=f"{name}-data"),
            location=FILE_URL,
        ),
    ],
    compute=SimpleComputeResource(
        volumes=[
            SimpleVolumeMount(
                resource=f"{name}-data",
                path="/input",
                mode="READONLY",
            ),
        ],
    ),
)

```



But in many cases, the end user would not need to create the execution description from scratch.

The developers of the software can supply a template, `ivoa-execution.yaml`, alongside their code in their git repository.

```
executable:
  kind: "http://purl/docker-container"
  image:
    locations:
      - "ghcr.io/zarquan/heliophorus-androcles"
    digest: "sha256:0dfe....4df3"
compute:
  kind: : http://purl/simple-compute-resource
  volumes:
    - kind: http://purl/simple-volume-mount
      path: /input
      mode: READONLY
      resource: example-data
data:
  - kind: http://purl/abstract-data-resource
    meta:
      name: "example-data"
```

The client can use the template to build an execution request by replacing the abstract data resource with a specific `simple-data-resource` that contains the location to download the data from.

Replacing this

```
data:
- kind: http://purl/abstract-data-resource
  meta:
    name: "example-data"
```

with this

```
data:
- kind: http://purl/simple-data-resource
  meta:
    name: "example-data"
  location: "http://.../example.dat"
```

Example of a template for a container:

<https://github.com/Zarquán/Heliophorus-androcles/blob/main/ivoa-execution.yaml>



Recap

ExecutionBroker is a set of endpoints that manage execution sessions.

- `/direct` Direct execution (POST, redirect)
- `/requests` Execution request (POST, redirect)
- `/offers/{uuid}` Offers response (GET)
- `/sessions/{uuid}` Session status (GET)

The simplest of which is `/direct`

POST the following task description to `/direct` and the service will execute it

`executable:`

`kind: "http://purl/jupyter-notebook"`

`notebook:`

`location: "http://.../example.jpnb"`

Add compute, data, and storage resources to create a more complex task.



Implementation progress

OpenAPI schema - done

- Java server components generated by GitHub workflow
- Python client generated by GitHub workflow
- Interoperable – using the Python client to test the Java service

Core engine – in progress

- Request validation - done
- Sequencing engine – in progress
- Resource booking – in progress

CANFAR platform, developed as part of SRCNet

- Execute docker containers - done
- Remote data download – todo
- Authentication token refresh - todo

Docker platform, developed as part of UKSRC

- Execute docker containers - done
- Remote data download – in progress
- Stress tests processing 1,000s of requests – done
- Bottle necks identified and performance fixes in progress

Metrics and costs

New features under development - metrics and costs.

Metrics are a simple way to describe what kind of compute performance is needed to execute a task.

For example, the the following metric requests a compute resource capable of reaching at least 3.5 on compute-benchmark-21.

```
compute:
  kind: "http://purl/simple-compute-resource"
  metrics:
    - kind: "http://purl/simple-min-max-metric"
      type: "http://purl/compute-benchmark-21"
      min: 3.5
```

The meaning of the value is specific to the type of benchmark.
Execution Broker is simply acting as the messenger.

Metrics

If the user wants a better level of performance than their current platform.

They can run the benchmark on their current platform, and then set that as the minimum criteria in their request.

```
compute:
  kind: "http://purl/simple-compute-resource"
  metrics:
    - kind: "http://purl/simple-min-max-metric"
      type: "http://purl/compute-benchmark-21"
      min: 3.5
```

We can create benchmarks for specific types of tasks,
e.g. cross matching, source extraction, image processing etc.

Each metric would match the mix of requirements for that type of task
e.g. IO bandwidth, floating point performance, GPU integration.

Metrics

If services include a benchmark metric in their responses, it gives the user a more meaningful way to compare the relative performance of different platforms.

```
compute:
  kind: "http://purl/simple-compute-resource"
  metrics:
    - kind: "http://purl/simple-min-max-metric"
      type: "http://purl/compute-benchmark-21"
      min: 4.1
```

```
compute:
  kind: "http://purl/simple-compute-resource"
  metrics:
    - kind: "http://purl/simple-min-max-metric"
      type: "http://purl/compute-benchmark-21"
      min: 6.2
```



Costs

Offering unlimited access to compute resources at zero cost doesn't scale well.

At least three different providers have mentioned that they have implemented, or are planning to implement, some kind of cost for using compute resources.

Not necessarily financial costs, but some kind of credits linked to the user's account that provide a way to measure how much compute they have used, and to encourage them to think about limiting how much they request.

```
compute:
  kind: "http://purl/simple-compute-resource"
  costs:
    - kind: "http://purl/simple-min-max-cost"
      type: "http://purl/user-account-credits"
      min: 5
```

Costs

Some of our colleagues at UCL have been looking at estimating the energy costs of running HPC compute resources and ways to estimate the carbon cost of executing tasks on different platforms.

```
compute:
  kind: "http://purl/simple-compute-resource"
  costs:
    - kind: "http://purl/simple-min-max-cost"
      type: "http://purl/basic-energy-cost"
      min: 9.1
    - kind: "http://purl/simple-min-max-cost"
      type: "http://purl/carbon-use-estimate"
      min: 22
```

Costs

Adding a costs element to the request and response enables the user and the service to have a conversation about how much a task will cost.

The user can specify a maximum cost they are willing to pay.

```
compute:
  kind: "http://purl/simple-compute-resource"
  costs:
    - kind: "http://purl/simple-min-max-cost"
      type: "http://purl/user-account-credits"
      max: 5
```

The service can declare the maximum and minimum cost to execute the task.

```
compute:
  kind: "http://purl/simple-compute-resource"
  costs:
    - kind: "http://purl/simple-min-max-cost"
      type: "http://purl/user-account-credits"
      min: 1
      max: 5
```

Costs

Declaring a minimum cost to execute a task may help to address some of the concerns people have about pilot jobs and resource blocking.

```
compute:  
  kind: "http://purl/simple-compute-resource"  
  costs:  
    - kind: "http://purl/simple-min-max-cost"  
      type: "http://purl/user-account-credits"  
      min: 1  
      max: 5
```

This declaration by the service means that the user's account will be charged at least one credit, just for accepting the offer.

Regardless of whether the task runs to completion or is canceled early.



Costs and metrics

Costs and metrics are for the future.

We are experimenting with them to see how useful they are.

They are part of the prototype, but they won't make it into the proposed standard until we understand more about how they work.

If you are interested in exploring this please join the discussion.



Back to basics

POST the following request to the /direct endpoint and the platform will run the container.

```
executable:  
  kind: http://purl/docker-container  
  image:  
    locations:  
      - "ghcr.io/zarquan/heliophorus-androcles"  
    digest: "sha256:0dfe....4df3"
```

It is up to each platform to choose how much they implement.

A simple implementation can choose to reject anything more complicated.

Thank you

Dave Morris
dave.morris@manchester.ac.uk



<https://github.com/ivoa-std/ExecutionBroker>

Please join the discussion.

We need your help to make it happen.